

Report

Numbers here come from `uv run brandbot eval --replay`. Nothing is typed by hand that the command did not print. Run it to check any figure in this file.

1. Problem framing

An inbound tweet at Hulu's support account needs three decisions: what is this about, what should the reply say, and can a person be skipped. This project builds that pipeline for one brand, `hulu_support`, and spends most of its effort proving whether it works rather than building more of it.

"Good" means, in order:

1. Nothing false goes out unedited. A confident wrong reply costs more than a slow right one, because a support lead has to apologise for it and re-answer.
2. Skip the human only when the message needed nothing but information. 64% of the golden set wants information alone, no account touched. That is the share of traffic worth automating at all.
3. Among what it answers, answer more of it correctly than copying the nearest past reply does. Retrieval alone is free; the agent has to earn its cost.

What I chose not to build, and why:

- **Multi-turn context.** 36.7% of messages in this corpus are follow-ups, and 39.7% of those ("still broken", "on Roku", "yes") cannot be understood without the thread. That is roughly one inbound message in seven. It was not built because there is no labelled follow-up anywhere in the golden set, all 220 are opening messages, so a thread-reading feature would ship with no evidence it helps, in a project whose whole thesis is that the proof matters more than the feature.
 - **A vector database or an orchestration framework.** The retrieval index is 9,859 vectors, 29MB, searched with one dot product in milliseconds. A hosted vector store earns its place past a million vectors, not here. An orchestration framework wraps the model-calling logic in someone else's abstractions, and the brief requires explaining and modifying this code live. Code I did not design end to end is harder to defend under questioning.
 - **A confidence-tuned escalation threshold.** The similarity floor is set once, by inspection, at 0.62. It fires on 0.9% of messages and changes nothing. Two raters agreeing on intent at kappa 0.53 but on routing at only 0.18 says the routing signal in this data is weak; tuning a threshold against noise that size would be fitting the noise, not the signal.
 - **A second escalation model per intent.** One content-rule set (`pipeline/policy.py`) decides escalation before any confidence score is consulted. A billing dispute goes to a person regardless of how good the draft looks. A learned per-intent policy would need labelled routing outcomes this project does not have, and would hide the cost-asymmetry judgement the content rules make explicit.
-

2. Results against two baselines

Three systems, all scored on the same 220 golden messages, all judged by the same rubric. **Majority** replies to everyone with the commonest intent's canned line and no retrieval. **Nearest** finds the closest past message by

embedding and sends whatever the brand actually replied then. **Agent** classifies, drafts a new reply grounded in the retrieved precedents, and decides the route.

System	Auto-sent	Of those, held back	Human touches / 100	Macro-F1	Same-next-step	Prior-weighted
agent	29%	6%	80	0.671 [0.573, 0.745]	0.777 [0.723, 0.827]	72%
nearest	99%	50%	249	0.450 [0.386, 0.503]	0.677 [0.614, 0.736]	48%
majority	100%	46%	230	0.032 [0.025, 0.041]	0.191 [0.141, 0.245]	23%

Auto-sent is coverage: the share answered with no human. **Held back** is risk: of what it sent, the share a support lead (via the judge) would have stopped before it reached the customer. Neither means anything alone: escalate everything and held-back goes to zero; send everything and coverage hits 100%. **Human touches per 100** combines them: one escalation costs one touch, one reply that should not have gone out costs five, covering the apology and the re-answer. Five is a stated assumption, not a measured cost for this brand; see section 4.

Two comparisons, paired on the same resampled examples so the shared difficulty of a given message cancels instead of inflating the gap:

- Agent vs nearest: macro-F1 +0.221 [0.126, 0.303], same-next-step +0.100 [0.027, 0.173]. Both intervals exclude zero.
- Agent vs majority: macro-F1 +0.639 [0.541, 0.713], same-next-step +0.586 [0.509, 0.655]. Both intervals exclude zero.

The agent beats both baselines and the gap survives resampling. But the more useful reading is the failure of the two baselines, because it says what the agent's job actually is. Nearest is worse than majority on held-back risk (50% vs 46%) despite having real content, because a genuine past reply commits to specifics: a name, a device, an error code, that are usually wrong for a new customer. Copying a real answer is not automatically safer than a generic one, it is differently unsafe. The agent's advantage is not "knows more", it is "escalates the 71% of messages retrieval and majority both auto-answer badly."

3. Top five failure modes

Real examples, drawn from the committed runs, not constructed for this section.

1. app_bug collapses into other under UI-redesign complaints. 25 of the agent's intent misses are on **service_outage**, **live_tv_channels**, or **app_bug**, and most of those are **app_bug** predicted as **other**: "Man, I had to turn off autoplay", "your new introduction is fucking ridiculous", "It's messier than before, especially with multiple people watching". These are complaints about a specific redesign choice, not bug reports, and the taxonomy's **app_bug** definition, written from crash and playback failures, does not cover them. Hypothesis: the taxonomy was induced by clustering on subject, and "the app changed and I hate it" reads, subject-wise, like nothing else in the set, so it falls into the catch-all. Fix would be a taxonomy change, not a classifier

change, and it needs enough labelled examples of this pattern to justify a new intent, which the current 220 do not provide (`app_bug` already has only 35).

2. `content_availability` and `live_tv_channels` are confused in both directions. "any chance of adding ABC to my Live subscription" (gold `live_tv_channels`) predicted `content_availability`; "please add #NFLRedZone ... to your live tv" (gold `live_tv_channels`) predicted `content_availability`. Both are requests to add a channel, which reads identically to "is this show available" unless the reader already knows Hulu's live-TV tier exists as a separate product. Hypothesis: the definitions were written from the same corpus's most common cases, and channel-addition requests are rare enough (10 of 220) that the taxonomy's boundary was never tested against them until now.

3. The judge and a human disagree in ways that lean toward the same blind spot. Of 27 agent replies rated blind, 3 disagreed with the judge. Two are the judge marking down a reply that "combines multiple past replies" as repetitive, when a human found it fine (thread 464784, 507575). The judge penalises a stitched-together draft that reads slightly redundant but is still correct, which is a tone objection dressed as a grounding one. The third (thread 520349) is the reverse: a human rejected a reply the judge passed for matching precedent phrasing, on a message about content appropriateness for kids watching with them, a judgement call about tone that the rubric's "grounded" question does not capture, because the reply was accurate and ungrounded is the wrong complaint. Hypothesis: "combines multiple precedents smoothly" is a genuine weakness in the current prompt template, worth a fix; the third is closer to a genuinely ambiguous case than a systematic gap.

A first rating pass over this same sample was thrown out. It came back 71/80 `send: true`, not a considered rating, and was re-rated from scratch. The numbers above are from the second, valid pass.

4. The nearest-neighbour baseline greets the wrong person by name. Six of its judge-failed replies address the customer as Jeff, Alaina, Mohammad, or Sean: names from the precedent conversation, wrong for the current one. This is the mechanism behind nearest's 50% held-back rate: copying a real reply copies every specific it committed to, including ones that do not transfer. The agent drafts fresh instead of copying verbatim and produced zero name errors in either run. This is the clearest evidence in the whole project for why grounding a new draft beats retrieving a finished one.

5. Escalation ground truth and the agent's escalation rules share an author. Route labels came from the same labelling pass that wrote the intents (see `docs/GOLDEN_SET.md`), and the agent's `policy.py` content rules were written by the same person, at a similar time, from the same read of the corpus. Sanyam's full review of all 220 labels is the only independent check on this. It means the 6% held-back figure for the agent is measuring consistency with one person's judgement about what should escalate, not an outcome anyone confirmed against real customer harm. This is not a bug to fix, it is a ceiling on what the routing number can honestly claim, and it is repeated in section 4 because it belongs there too.

4. What is about the headline number

The headline is "agent macro-F1 0.671, 6% held-back, 80 human touches per 100." Six things narrow what that actually means.

The intent labels were model-proposed, human-reviewed All 220 were first initially labelled by Claude applying the frozen taxonomy definitions, then reviewed and altered by Sanyam in the labelling tool with every field overridable (`docs/GOLDEN_SET.md`). Macro-F1 against this file partly measures agreement between

the agent's classifier and another language model's reading of the same rules, not raw correctness. 0.671 macro-F1 sits above that ceiling only because it is measured against the reviewed file.

The golden set is stratified, so its intent mix is not real traffic's mix. 70 of 220 examples were drawn specifically to give thin intents enough examples to score. Macro-F1 answers "how good is each intent, weighted equally"; prior-weighted accuracy (72%, computed only from the 150 uniformly drawn examples) answers "what fraction of tomorrow's unsorted traffic lands correctly." They differ by design and both are quoted for that reason. Neither alone is the number.

The touches-per-100 figure depends on a cost ratio that was never measured for this brand, and the ranking is not stable under it. Five human touches per bad auto-reply is a conventional placeholder in the code (`report.py`, `COST_RATIO`), not a figure Hulu's support team confirmed. Running the same command with `--cost-ratio 1`, a bad reply costs the same as one escalation, flips the ranking: agent 73 touches/100, nearest 50, majority 46. The agent escalates 71% of the time, and when a bad auto-reply is cheap, escalating that often costs more than just answering and eating the occasional bad reply. Only past roughly cost-ratio 2 does the agent's low risk start winning back the touches it spends on escalation (at ratio 5, agent 80 vs nearest 249). So the headline "80 touches per 100" is not a fact about the agent. It is a fact about the agent **given a guess about how expensive a mishandled ticket is**, and a different, equally defensible guess reverses which system to ship.

220 examples is small, and the intervals say so. Agent macro-F1's interval spans 0.573 to 0.745, nearly a fifth of the scale. The agent-vs-nearest gap clears that noise (both bootstrap intervals exclude zero), but a narrower true gap on a bigger set could easily have not.

The golden set is the newest slice of time, and the retrieval index is systematically older. Splits are chronological so retrieval never sees the future, which means the precedents the agent cites are, on average, from earlier in Hulu's support history than the messages being scored. Any product, policy, or support-style drift between those periods works against the agent and is baked into every number here, not corrected for.

The judge agrees with the human it was checked against at kappa 0.378 overall, "fair" on the interpretation scale, the middle band, not strong agreement. This is the single most important qualifier in the report and it belongs here as much as in section 3. Per system it splits: nearest 0.562 (moderate), agent 0.341 (fair, but the interval is wide, [-0.110, 1.000] on $n=27$, so this one is close to uninformative on its own), majority 0.014 (slight, on only 40.7% raw agreement: the judge and the human disagree on most of majority's replies). Majority's low number is worth reading rather than dismissing: majority sends the same canned line regardless of what was asked, and the judge and a human evidently draw the line on "would you send this to this specific customer" in different places once the reply stops being tailored to the message at all. **Every "held-back" and "risk" number in section 2 is downstream of the judge**, and this result says the judge is usable, better than a coin flip, nowhere near a substitute for a person, which qualifies those numbers without discarding them.

A first rating pass over this same 80-reply sample was thrown out before these figures were computed: it came back 71/80 `send: true`, which is not a considered judgement, and was re-rated from scratch rather than kept. State that here as much as the kappa itself, because a rushed human rating would have produced a falsely rosy agreement figure. A judge that also mostly says "send" would have looked like it agreed. The fix was to redo the rating, not to adjust the number.

5. With one more week

In priority order, because the first two are what the numbers above actually call for and the rest are what would matter next.

1. **Rate another 150-200 replies.** The judge-human agreement figure is the least trustworthy number in this report because it rests on 27 examples per system, 81 total. This is the single highest-value next step: everything else in section 2 is qualified by how much to trust the judge.
 2. **Fix the `app_bug/other` boundary.** The most common single failure mode found in section 3 is mechanical: the taxonomy has no category for "I hate this redesign," and it is common enough (part of `other`'s 17% of the set) to be worth a new intent, not a prompt patch.
 3. **Cross-judge a sample with a model from a third family,** as `docs/EVALUATION.md` already specifies and the code does not yet run. The agent and the judge are both drawn from adjacent model families; a same-family judge rewards familiar phrasing in a way a human would not.
 4. **Get a real cost ratio from someone at Hiver or from public support-cost figures,** instead of the placeholder 5. It does not change the ranking here, but it would let the touches-per-100 number make a claim about actual operating cost rather than a relative one.
 5. **Label a set of follow-up messages** and measure whether reading the thread changes anything, before building the feature. Decision 18 in `docs/DECISIONS.md` explains why this was skipped rather than built unevaluated; a week is enough to label the ~30 examples needed to find out if it is worth doing at all.
-

6. Citations

- Dataset: [Customer Support on Twitter](#) by Stuart Axelbrooke, via Kaggle. Terms in `data/LICENSE`.
- Cohen's kappa interpretation bands: Landis, J.R. and Koch, G.G. (1977), "The Measurement of Observer Agreement for Categorical Data," *Biometrics* 33(1). Conventional and arbitrary, stated as such in `eval/agreement.py`.
- Embedding model: `nomlc-embed-text`, served locally through [Ollama](#). Chosen over a hosted metered endpoint for speed and to avoid a quota; see decision 7 in `docs/DECISIONS.md`.
- Libraries: `typer`, `rich`, `numpy`, `scikit-learn` (for `cohen_kappa_score`), `pytest`. No orchestration framework, no vector database, see decision 19.
- Intent taxonomy labels for `data/interim/reference_labels.jsonl` first pass: written by Claude (Anthropic), applying the frozen taxonomy definitions and routing spec I wrote; reviewed and accepted by me. Full provenance in `docs/GOLDEN_SET.md`.
- I used an AI coding assistant (Claude, via Claude Code) throughout this project for implementation, debugging, and drafting documentation, under my direction and review. Design decisions (the brand-selection criterion, the chronological split, the taxonomy merge, the escalation rules, what to leave out) are mine, recorded with reasoning in `docs/DECISIONS.md` so they can be defended without the assistant present.